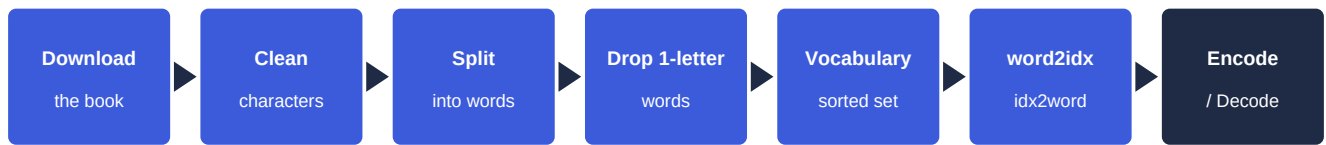


Preparing Real Text for Tokenization

Lesson 02, Preparing_text notebook: from a whole book to token IDs



1. The idea in simple words

The first notebook used three tiny sentences. Real text is messy: it has odd symbols, numbers, punctuation, capital letters and line breaks. A computer model needs **clean, uniform pieces** before it can turn them into numbers. This notebook takes a real book, *The Time Machine* by H. G. Wells (free from Project Gutenberg), cleans it, chops it into words, builds a word list (the **vocabulary**), and gives every word a number (its **token ID**). Then it proves the process works by turning numbers back into words.

2. What each step does

Step	What happens (simple version)	Code idea
1. Get the book	Download the full book as one very long string of text. It has 179,799 characters.	<code>requests.get(url).text</code>
2. Remove odd characters	Replace strange symbols (curly quotes, long dashes, underscores) with a space.	<code>re.sub(...)</code>
3. Clean further	Delete all non-English-keyboard characters and all digits, then make everything lowercase so <i>The</i> and <i>the</i> match.	<code>.lower()</code>
4. Split into words	Cut the text at every punctuation mark or whitespace. Throw away empty pieces.	<code>re.split(...)</code>
5. Drop 1-letter words	Keep only words longer than one letter (removes leftovers like <i>s</i> , <i>h</i> , <i>g</i>).	<code>len(item)>1</code>
6. Make the vocabulary	List each different word once, in alphabetical order: 30,699 words become 4,589 unique tokens.	<code>sorted(set(words))</code>
7. Make two lookup tables	word2idx : word to number. idx2word : number to word.	dictionary comprehensions
8. Encode and decode	Turn words into a NumPy array of IDs, and IDs back into a sentence.	<code>encoder()</code> / <code>decoder()</code>

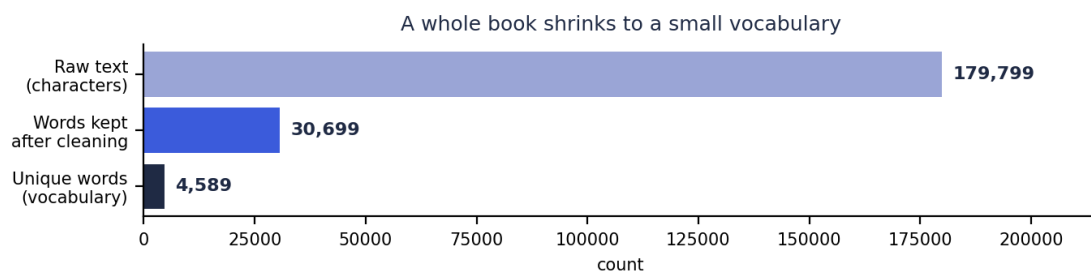
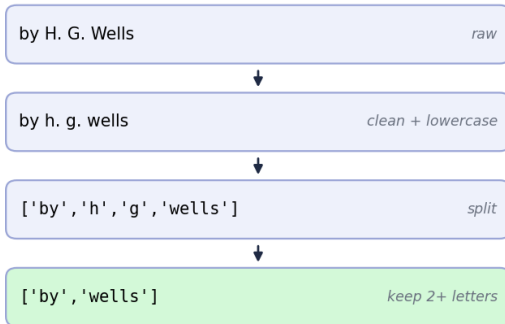


Figure 1. Real counts from the notebook. Many repeated words collapse into one vocabulary entry, so the model needs far fewer numbers than there are words.

3. Cleaning, shown with real examples

The notebook has two cleaning passes. The first list (`strings2replace`) targets text that was wrongly decoded, where a curly quote turns into junk like `â\x80\x9c`. In the printed output of this run the quotes and dashes already looked normal, so the second pass did the real work: `re.sub(r'^\x00-\x7F|+', ' ', text)` swaps every non-ASCII character for a space. That is simple and safe, but it has side effects you can see in the figure below.

Example 1



Example 2

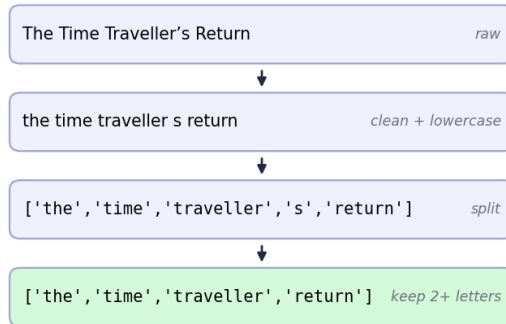


Figure 2. Two real cases. H. G. becomes two stray letters that the 1-letter filter removes. The apostrophe in Traveller's becomes a space, so the word splits and the stray s is dropped.

4. Vocabulary and IDs

The IDs are only **positions in the alphabetical list**. abandon is 0 because it sorts first, and wonderful is near the end (4524). The notebook shows this by decoding IDs 1, 3 and 10, which gives *abandoned abnormally absent*: three words that start with *ab*. A bigger number does not mean a more important word.

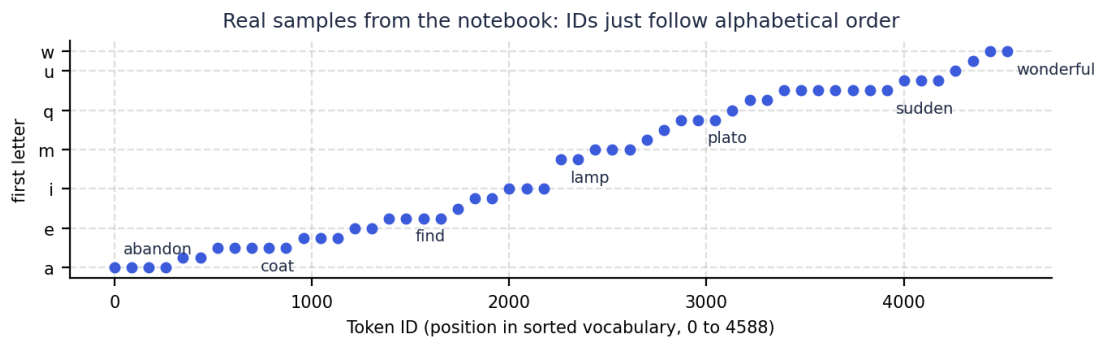


Figure 3. Each dot is a real (word, ID) pair printed by the notebook (every 87th entry). IDs climb steadily with the alphabet.

5. Encoder and decoder

```

def encoder(words, encode_dict):
    idxs = np.zeros(len(words), dtype=int)    # empty array of integers
    for i, w in enumerate(words):
        idxs[i] = encode_dict[w]              # look each word up
    return idxs

def decoder(idxs, decode_dict):
    return ' '.join([decode_dict[i] for i in idxs]) # numbers back to a sentence

encoder(['the', 'time', 'machine'], word2idx)    # [4042 4109 2416]
  
```

- Compared with lesson 1, the lookup tables are now **passed in as arguments**, so the same functions work with any vocabulary.
- The encoder returns a **NumPy array** (ready for later maths); the decoder returns one **joined string**.
- Test:** pick a random start, take 10 words in a row, encode, decode. The decoded text matched the original words exactly.

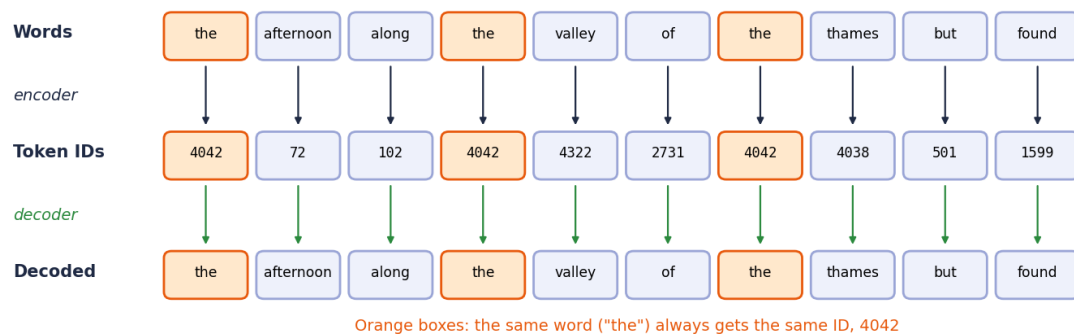


Figure 4. The round trip from the notebook (random start at word 19,284). Every "the" maps to 4042 and comes back as "the".

6. Things worth noticing

What to notice	Why it matters	Simple fix to try later
Apostrophes split words (<i>traveller s</i>)	Contractions and possessives lose their ending	Keep apostrophes inside words in the split pattern
1-letter filter also removes real words (<i>a, i</i>)	The model never sees them	Remove only specific junk letters, or skip the filter
Book header, chapter numerals stay (<i>start of the project gutenber</i> <i>g, ii, xv</i>)	Non-story words enter the vocabulary	Cut the text at the start and end markers
Patterns like <code>'\d+'</code> and <code>'[...\s]+'</code> are not raw strings	Python printed a SyntaxWarning in the notebook	Write them as <code>r'\d+'</code> and <code>r'[...\s]'</code>
Unknown word means KeyError	The vocabulary only knows this one book	Add an unknown-word token
Random start index	Output differs each run	Set <code>np.random.seed(...)</code>

Takeaway: before a model sees any text, we **clean it, split it, list its unique words, and number them**. Every choice (lowercasing, punctuation, filters) changes the vocabulary, so these small decisions quietly shape what the model can learn. Real LLM tokenizers follow the same recipe with smarter splitting.